Universidad Nacional Autónoma de México
Facultad de Ingeniería
Bases de Datos

Tarea 8
Modelo Entidad-Relación Extendido (MERE)

García Olvera José Ignacio
30 de septiembre de 2026

Introducción

El Modelo Entidad-Relación Extendido (MERE) amplía el modelo entidad-relación básico para casos donde varias entidades comparten atributos, o donde una relación debe conectarse con otra entidad. Su idea principal se parece a la herencia en la programación orientada a objetos. Se apoya en dos conceptos:

- **Supertipo:** entidad que concentra los atributos comunes (por ejemplo, `Persona`).
- **Subtipo:** entidad con los atributos particulares de un rol (por ejemplo, `Profesor`). Debe cumplirse la regla del *ES*: un profesor *es* una persona. Cada subtipo tiene al menos un atributo propio y no repite los comunes.

1. Generalización

Concepto. Es un proceso *bottom-up* (de abajo hacia arriba). Se parte de dos o más entidades ya existentes, se identifican sus atributos en común y se extraen a una nueva entidad supertipo.

Ejemplo. Se tienen tres entidades con atributos repetidos:

- `Profesor`(nombre, ap_paterno, ap_materno, edad, email, rfc, especialidad)
- `Investigador`(nombre, ap_paterno, ap_materno, edad, email, num_cedula, total_articulos)
- `Administrativo`(nombre, ap_paterno, ap_materno, edad, departamento)

Los atributos comunes (nombre, apellidos y edad) se extraen a `Persona`(persona_id, nombre, ap_paterno, ap_materno, edad). Los subtipos conservan solo lo particular: `Profesor`(email, rfc, especialidad), `Investigador`(email, num_cedula, total_articulos) y `Administrativo`(departamento), todos identificados por `persona_id`.

Ventajas (al llevarla a una sola tabla, con los atributos de los subtipos dentro de `Persona`):

- Mejor desempeño: no requiere *joins* entre tablas.
- Adecuada cuando cada subtipo tiene muy pocos atributos particulares.
- Los atributos comunes se definen una sola vez.

Desventajas:

- Los atributos particulares deben ser opcionales (`NULL`), pues no toda persona es profesor, investigador o administrativo.
- Menor control de integridad: la base de datos permite registrar un profesor sin su RFC.

2. Especialización

Concepto. Es un proceso *top-down* (de arriba hacia abajo). Se parte de una sola entidad que tiene atributos que solo tienen sentido para ciertas instancias (roles), y se crean subtipos con esos atributos.

Ejemplo. La entidad `Persona` tiene los campos rfc, especialidad, num_cedula, total_articulos y departamento, todos `NULL` porque dependen del rol. Esos campos se separan en las tablas `Profesor`, `Investigador` y `Administrativo`, cada una ligada a `Persona` mediante `persona_id`. El resultado es el mismo esquema del ejemplo anterior, pero construido en sentido inverso.

Ventajas:

- Los atributos particulares pueden definirse como `NOT NULL`, con mayor control de la integridad de los datos.
- Cada clase de una aplicación orientada a objetos corresponde directamente a una tabla.
- Adecuada cuando supertipo y subtipos tienen una cantidad similar de atributos.

Desventajas:

- A más niveles de jerarquía, más *joins* (para obtener un contador se unen `contador`, `administrador` y `persona`); se recomienda un máximo de 2 niveles.
- Si se elimina el supertipo y los atributos comunes pasan a cada subtipo, esos datos se repiten cuando una persona tiene varios roles.

Restricciones de la jerarquía

La relación entre supertipo y subtipo se describe con dos restricciones: *excluyente* (un solo subtipo) o de *traslape* (varios), y *total* (al menos un subtipo) o *parcial* (puede no tener ninguno). La cardinalidad entre supertipo y cada subtipo siempre es (0,1). Para saber a qué subtipo pertenece una instancia se usa un **discriminante**:

Combinación	**Significado**	**Discriminante**
Excluyente / total	Un solo rol, obligatorio	`tipo CHAR(1) NOT NULL`
Excluyente / parcial	A lo más un rol	`tipo CHAR(1) NULL`
Traslape / total	Varios roles, mínimo uno	Banderas booleanas `es_profesor`, `es_admin`, `es_investigador`
Traslape / parcial	Cero o varios roles	Las mismas banderas, `NOT NULL`

3. Relaciones de agregación

Concepto. Se presenta cuando existe una relación M:N entre dos entidades A y B, y una tercera entidad C debe relacionarse con *el resultado* de esa relación, no solo con A o con B. En el diagrama conceptual

la relación M:N se encierra en un cuadro y se trata como si fuera una nueva entidad. En el diseño lógico, C se relaciona con la tabla intermedia que produce la relación M:N.

Ejemplo. En una feria de empleo, las empresas entrevistan a solicitantes. `Empresa` y `Solicitante` tienen una relación M:N llamada `Entrevista`, que guarda la calificación y las observaciones. Cuando un solicitante es aceptado se genera una `Oferta_trabajo`, y esta debe asociarse a la entrevista que lo acreditó. Diseño lógico:

- `Empresa`(empresa_id, nombre, descripcion)
- `Solicitante`(solicitante_id, curp, nombre, cv, cedula)
- `Entrevista`(entrevista_id, calificacion, observaciones, empresa_id (FK), solicitante_id (FK))
- `Oferta_trabajo`(oferta_trabajo_id, num_contrato, fecha_oferta, tel_contacto, email_contacto, entrevista_id (FK))

Cada oferta proviene de exactamente una entrevista, y una entrevista genera cero o una oferta.

Ventajas:

- La oferta no duplica datos de la empresa ni del solicitante: los obtiene a través de la entrevista.
- Garantiza que solo exista una oferta para una pareja empresa-solicitante que realmente tuvo una entrevista.
- Representa con fidelidad lo que ocurre en el problema real.

Desventajas:

- El diagrama y el modelo resultan más complejos de leer.
- Consultar todos los datos de una oferta requiere varios *joins* (oferta, entrevista, empresa y solicitante).
- La relación M:N debe convertirse en una tabla con llave propia para poder relacionarla.

4. Atributos con histórico

Concepto. Algunos atributos cambian con frecuencia (por ejemplo, el estatus de una orden de compra) y, por control, se desea guardar cada cambio junto con su fecha. Al diseñarlos se cumplen dos reglas: almacenar el **valor actual** del atributo y conocer su **historial de cambios** a lo largo del tiempo. Los estados posibles se guardan en un *catálogo* (por ejemplo, `Status_orden_compra` con clave, descripción y activo). Otros ejemplos son los estados de un vuelo, de un préstamo de biblioteca o las mediciones de presión arterial de un paciente.

Ejemplo. Una orden de compra puede estar REGISTRADA, PAGADA, CANCELADA o ENTREGADA. Entre `Orden_compra` y el catálogo de estatus se crean dos relaciones:

- **Estatus actual (1:m):** una orden tiene exactamente un estatus actual (1,1); un estatus puede asignarse a cero o muchas órdenes. La fecha del estatus también se guarda en la orden.
- **Histórico (m:m):** una orden tiene al menos un estatus y hasta N a lo largo del tiempo; un estatus puede asignarse a cero o muchas órdenes. Esta relación guarda la fecha de cada estatus.

Diseño lógico:

- `Orden_compra`(orden_compra_id, folio, tipo, fecha_status, status_orden_compra_id (FK))

- `Status_orden_compra`(status_orden_compra_id, clave, descripcion, activo)
- `Historico_status_orden`(historico_status_orden_id, fecha_status, orden_compra_id (FK), status_orden_compra_id (FK))

Al crear una orden se registra su estado inicial y se agrega una entrada al histórico. Cuando el estado cambia, se actualizan `status_orden_compra_id` y `fecha_status` en la orden y se agrega una nueva entrada al histórico para no perder los valores anteriores. Datos de muestra (el estado actual también aparece en el histórico):

ORDEN_COMPRA

orden compra id	status orden compra id	fecha status	folio	tipo
1	3	22-03-2023 10:00:08	892893	O

HISTORICO_STATUS_ORDEN

historico status orden id	status orden compra id	fecha status	orden compra id
1	1	21-03-2023 09:35:40	1
2	2	21-03-2023 09:54:15	1
3	3	22-03-2023 10:00:08	1

Ventajas:

- El estado actual se consulta directamente en la orden, sin recorrer el histórico (que suele ser muy grande); el histórico se consulta solo cuando se necesitan valores anteriores.
- No se pierden los estados anteriores ni las fechas en que cambiaron.
- Los estados se definen una sola vez en el catálogo.

Desventajas:

- Existe redundancia: la relación 1:m y la fecha de la orden podrían eliminarse porque sus valores ya están en el histórico. Se dejan a propósito para mejorar el desempeño.
- Cada cambio de estado requiere dos operaciones: actualizar la orden y agregar una entrada al histórico.

Conclusión

La generalización y la especialización permiten organizar atributos comunes y particulares mediante supertipos y subtipos; la primera parte de entidades existentes y la segunda de una entidad que se divide. La elección entre ellas depende de cuántos atributos compartan las entidades y de cuánta integridad se requiera. La agregación, por su parte, permite relacionar una entidad con otra relación completa. Por último, el manejo de atributos con histórico conserva tanto el valor actual como todos los cambios anteriores de un atributo, mediante un catálogo y dos relaciones.

Referencia

Rodríguez C., J. A. *5. Modelo de Datos Extendido.* Material de apoyo, Bases de Datos, Facultad de Ingeniería, UNAM.